What is a Pod YAML?

Simple Definition

A Pod YAML is a configuration file that tells Kubernetes how to create a Pod.

Think of it like a recipe.

- **Recipe → Ingredients + Instructions**
- **YAML → Pod configuration + Instructions**

Simple Pod YAML Example

Create a file named pod.yaml

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

Explain Each Line

1. API Version

apiVersion: v1

- **Specifies the Kubernetes API version.**

- **v1 is used for Pods.**

---

2. Kind

kind: Pod

- **Tells Kubernetes what you want to create.**
- **Here, we're creating a Pod.**

---

3. Metadata

metadata:

name: nginx-pod

- **Stores information about the Pod.**
- **name is the Pod's name.**

Pod Name:

nginx-pod

---

4. Spec

spec:

- **Defines how the Pod should run.**
- **Contains the container configuration.**

---

5. Containers

containers:

- **List of containers inside the Pod.**

---

6. Container Name

- name: nginx-container

- **Name of the container.**

---

7. Image

image: nginx

- **Docker image to use.**
- **Kubernetes downloads the nginx image from Docker Hub.**

---

8. Port

ports:

- containerPort: 80

- **Opens port 80 inside the container.**

Visual Diagram

Pod

│

├── Name: nginx-pod

│

└── Container

│

├── Name: nginx-container

├── Image: nginx

└── Port: 80

How to Create the Pod

Step 1: Create the YAML file

vi pod.yaml

Paste the YAML and save it.

Step 2: Create the Pod

kubectl apply -f pod.yaml

Output:

pod/nginx-pod created

Step 3: Check the Pod

kubectl get pods

Output:

NAME READY STATUS RESTARTS AGE

nginx-pod 1/1 Running 0 20s

Step 4: View Detailed Information

kubectl describe pod nginx-pod

Step 5: View Logs

kubectl logs nginx-pod

Step 6: Delete the Pod

kubectl delete -f pod.yaml

or

kubectl delete pod nginx-pod

Important YAML Keywords

Keyword	Meaning
apiVersion	Kubernetes API version
kind	Type of resource (Pod, Deployment, Service)
metadata	Information about the resource
name	Name of the Pod
spec	Desired configuration
containers	List of containers
image	Container image
ports	Container port

KCNA Memory Trick ⭐

apiVersion → Which API?

kind → What to create?

metadata → Resource information

spec → How should it run?

containers → What runs inside?

image → Which application?

ports → Which port?

Complete Flow

Write pod.yaml

|

▼

kubectl apply -f pod.yaml

|

▼

Kubernetes creates the Pod

|

▼

Pod starts the Container

|

▼

Application starts running

⭐ **Most Important Commands**

kubectl apply -f pod.yaml # Create Pod

kubectl get pods # List Pods

kubectl describe pod nginx-pod # Pod details

kubectl logs nginx-pod # View logs

kubectl delete -f pod.yaml # Delete Pod

These are the core Pod YAML concepts and commands that every KCNA beginner should know.

Why is YAML Used in Kubernetes?

Simple Definition

YAML is a human-readable configuration file used to tell Kubernetes what to create and how to create it.

Beginner Definition

YAML is like an instruction sheet or blueprint that tells Kubernetes exactly how your application should run.

Real-Life Example 🏠

Imagine you want to build a house.

You don't tell the workers everything every day.

Instead, you give them a **blueprint**.

Blueprint

↓

Workers

↓

House

Similarly,

YAML File

↓

Kubernetes

↓

Pod

The YAML file is the **blueprint** for Kubernetes.

Why Not Create Pods Manually?

Imagine you want to create **100 Pods**.

Without YAML, you would have to type the command 100 times.

Example:

kubectl run nginx --image=nginx

kubectl run nginx2 --image=nginx

kubectl run nginx3 --image=nginx

...

This is:

- ❌ Time-consuming
- ❌ Difficult to manage
- ❌ Easy to make mistakes

With YAML:

kubectl apply -f pod.yaml

Everything is created from one file.

Why Do Companies Use YAML?

Large companies may have:

- 100 Pods
- 200 Services
- 50 Deployments
- 30 ConfigMaps

It is impossible to remember every command.

Instead, they save everything in YAML files.

Benefits:

- Easy to edit
- Easy to share
- Easy to reuse
- Easy to store in GitHub
- Easy to recreate the environment

What Information Can YAML Store?

A YAML file can define almost every Kubernetes resource.

Examples:

- Pod
- Deployment
- Service
- Namespace
- ConfigMap
- Secret
- Job
- CronJob
- StatefulSet
- DaemonSet
- Ingress
- PersistentVolume
- PersistentVolumeClaim

So YAML is **not only for Pods**.

What Other Methods Are Used to Create Pods?

There are **three common ways**.

Method 1: Imperative Commands (CLI)

Create a Pod directly using a command.

Example:

kubectl run nginx --image=nginx

Advantages

- Very fast
- Good for testing
- Easy for beginners

Disadvantages

- Hard to manage large projects
- Not reusable
- Not recommended for production

Method 2: YAML Files (Declarative) ⭐⭐⭐⭐⭐

Example:

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
  containers:
  - name: nginx
    image: nginx
```

Create it with:

```
kubectl apply -f pod.yaml
```

Advantages

- Reusable
- Easy to edit
- Easy to store in GitHub

- Best practice
- Used in companies

Method 3: Helm Charts (Advanced)

Helm uses YAML templates to deploy complex applications.

Example:

helm install wordpress

Helm automatically creates many Kubernetes resources.

This is an advanced topic, so you don't need to worry about it yet.

Imperative vs Declarative

Imperative (Tell Kubernetes HOW)

kubectl run nginx --image=nginx

You are telling Kubernetes exactly **what to do now**.

Declarative (Tell Kubernetes WHAT you want)

kind: Pod

image: nginx

Then:

kubectl apply -f pod.yaml

You describe the desired state, and Kubernetes figures out how to achieve it.

Why is YAML Better?

Imagine tomorrow your server crashes.

Without YAML:

You must remember every command.

😢 Very difficult.

With YAML:

kubectl apply -f pod.yaml

Everything comes back.

This is called **Infrastructure as Code (IaC)**.

What Can We Configure Inside a Pod YAML?

Many things!

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

You can configure:

- Pod name
- Labels
- Namespace
- Container image
- Port

- Environment variables
- CPU and Memory
- Storage (Volumes)
- Secrets
- ConfigMaps
- Restart policy
- Health checks (Liveness & Readiness Probes)
- Security settings

Easy Comparison

Command	**YAML**
Fast	Slightly slower to write
Temporary	Permanent
Hard to reuse	Easy to reuse
Good for testing	Good for production
Short	More detailed
Not stored	Can be stored in GitHub

Real Company Workflow

Developer

|

▼

Writes YAML File

|

▼

Stores it in GitHub

|

▼

CI/CD Pipeline

|

▼

kubectl apply -f pod.yaml

|

▼

Kubernetes

|

▼

Pod Created

This is how most companies deploy applications.

Quick Revision

Why is YAML Used?

- 📄 Human-readable configuration file.
- 📦 Creates Kubernetes resources.
- ♻️ Reusable.
- 📝 Easy to modify.
- 📂 Stored in GitHub.
- 🚀 Used in production.
- 🤖 Supports Infrastructure as Code (IaC).

Ways to Create a Pod

Method	Used For
kubectl run	Quick testing and learning
YAML (kubectl apply -f)	Production and best practice ⭐
Helm	Advanced application deployment

Easy Memory Trick ⭐

Think of it this way:

Restaurant 🍕

Customer

|

▼

Menu Card (YAML)

|

▼

Chef (Kubernetes)

|

▼

Pizza (Pod)

Without the **menu card (YAML)**, the chef won't know exactly what to prepare.

Similarly:

- 📄 **YAML = Instructions/Blueprint**
- ☸️ **Kubernetes = Chef**
- 📦 **Pod = Final Product**

⭐ KCNA Interview Question

Q: Why is YAML preferred over kubectl run?

Answer:

- YAML is reusable.
- YAML is version-controlled (stored in Git).
- YAML is easy to edit and share.
- YAML supports Infrastructure as Code (IaC).
- YAML is the standard method used in production Kubernetes environments.